

Table des matières

Rapport - Labo sans fil	1
Introduction	1
Manipulations et observations	2
Visualisation TTN et identification des gateways LoRaWAN	2
Test de simulation des données	2
Test et optimisation d'un décodeur JavaScript TTN	3
Accès au conteneur ALPHANET-DS	4
Intégration Webhook HTTP dans ALPHANET-DS	4
Affichage des données	6
Gestion des alertes	6
Webhook JSON vs Protocol Buffers	7
Conclusion	7
Reponse aux questions du compte-rendu	9
1. Relier deux LAN Ethernet via un bridge WIFI	9
b) Consultation du format de trame 802.11, explication de comment les 4 adresses devraient être utilisées, si cette fonction n'était pas activée	9
2. exemple de secret partagé qui permette d'atteindre 256 bits	10
3. qu'est-ce qu'un portail captif dans le contexte de la sécurisation d'un réseau sans-fil sans chiffrement ? y-a-t-il des problèmes avec cette stratégie ?	11
4. faites un schéma couche 3 et couche 2 du pont WiFi dans la variante où les équipements réseau sont administrés dans un VLAN séparé, placez un PC d'administration et indiquez quels ports sont en trunk et quel port est en access	11
5. avec deux antennes à gain 30 dB, quelle distance pourriez-vous atteindre ? quel serait le problème de cette liaison ?	11
6. donnez des arguments pour l'exploitation en mode bridge et en mode routeur au sein de l'entreprise (les réseaux filaires et sans-fils sont respectivement switchés et routés)	13
Bridge	13
7. Problème: des trames se perdent	13
8. Améliorer la sécurité d'un réseau de capteurs codés en 802.11 dont le chip wifi ne supporte pas le chiffrement en couche 2	13
9. Listez les mesures à prendre pour sécuriser un réseau Wifi et classez-les de la meilleure à la moins utile.	14
Annexes	14
Utilisation de l'IA	14
Sources	14

Rapport - Labo sans fil

Introduction

Dans ce laboratoire, nous nous sommes intéressés à la transmission et au traitement de données dans un contexte IoT à l'aide du réseau LoRaWAN et de la plateforme TTN (The Things Network).

L'objectif était de comprendre le fonctionnement complet d'une chaîne de communication, depuis l'envoi de données par un capteur jusqu'à leur réception, leur décodage et leur exploitation sur un serveur. Nous avons notamment observé les messages uplink, mis en place un décodeur de payload, puis configuré un webhook HTTP afin de recevoir les données dans un conteneur.

Nous avons ensuite développé un script PHP permettant de traiter, stocker et afficher les données reçues, ainsi que d'identifier des situations anormales. Enfin, nous avons comparé différents formats de transmission (JSON et Protocol Buffers) afin d'évaluer leur impact sur l'exploitation des données.

Ce laboratoire s'inscrit dans la continuité des notions vues en réseau et en développement web, et permet d'illustrer concrètement le fonctionnement d'une architecture IoT complète, depuis la communication sans fil jusqu'au traitement côté serveur.

Manipulations et observations

Visualisation TTN et identification des gateways LoRaWAN

Après l'authentification sur le site TTN, nous sommes arrivés sur le dashboard de notre capteur LoRaWAN. Nous avons sélectionné l'onglet "Live data" du dashboard, puis appuyé sur le bouton du capteur afin de visualiser notre première donnée transmise.

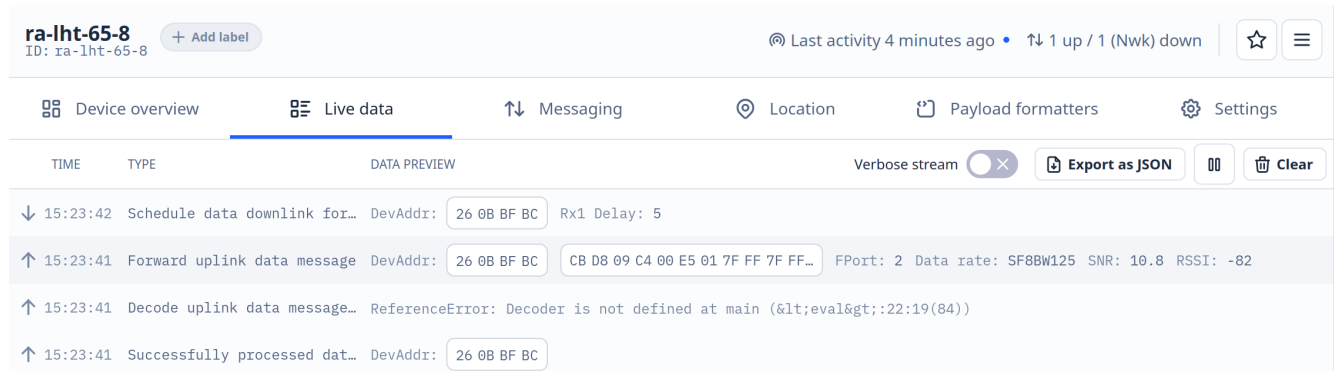


Figure 1: Première donnée transmise

Nous observons les étapes de traitement de l'uplink: 1. Forward uplink: réception du message 2. Decode uplink: tentative de décodage (échoue, pas de decodeur) 3. Schedule downlink: éventuelle réponse planifiée 4. Successfully processed: fin du traitement

En cliquant sur une de ces étapes, cela ouvre le détail de cette dernière au format JSON. C'est ici que nous pouvons voir par quel gateway notre message passe.

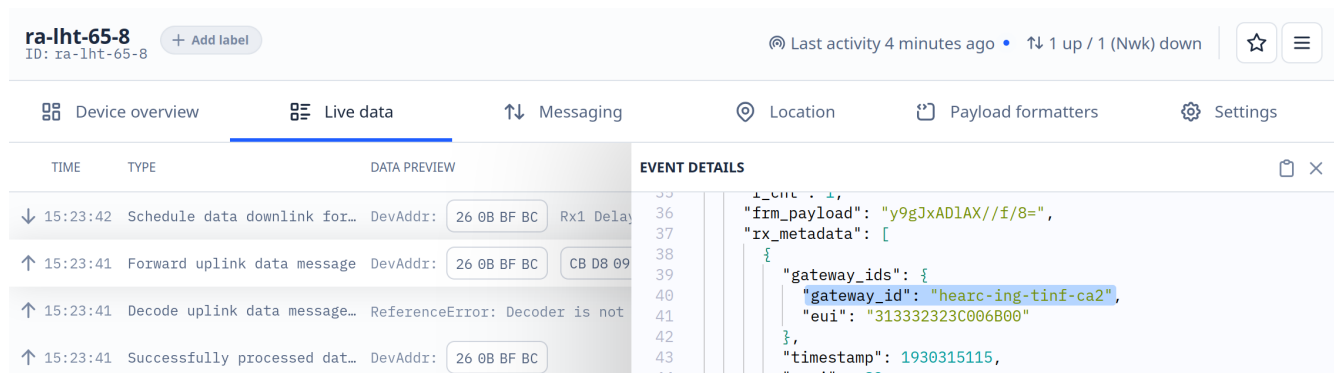


Figure 2: Gateway dans le détail de l'étape

Dans notre cas, le message uplink est transmis du capteur vers le serveur applicatif via le gateway HE-ARC.

L'identification du gateway montre le rôle de ces équipements comme interfaces entre le réseau LoRa (couche physique) et Internet (couche IP).

Test de simulation des données

Dans l'onglet "Messaging", puis la section "Simulated uplink" du dashboard, nous pouvons simuler un message qui pourrait être envoyé par le capteur. C'est pratique quand l'appareil physique n'est pas à portée de main.

Ensuite, nous pouvons vérifier dans l'onglet "Live data" que nous recevons bien un message, comme s'il provenait du capteur.

[Schedule downlink](#) [Simulate uplink](#)

Simulate uplink

FPort*

1

Payload

CC 54 08 0D 01 78 01 7F FF 7F FF

The desired payload bytes of the uplink message

Simulate uplink

Success

Uplink sent

Figure 3: Exemple de simulation d'uplink

Test et optimisation d'un décodeur JavaScript TTN

Nous avons ensuite mis en place un décodeur afin de "décrypter" les messages en hexadécimal. Nous sommes repartis de l'exemple fourni du cours.

Avec cette base, les messages en hexadécimal deviennent un format JSON qui nous retourne des informations sur la batterie:

ra-lht-65-8
ID: ra-lht-65-8

+ Add label

Last activity 7 minutes ago • ↑ 2 up / 1 (Nwk) down

☆

≡

Test

Byte payload

CC 54 08 0D 01 78 01 7F FF 7F FF

FPort

1

Test decoder

Decoded test payload

```
{
  "data": {
    "battery_status": 3,
    "battery_voltage": 3156
  }
}
```

Figure 4: Résultats du décodeur de base

Puis nous l'avons complété en suivant le commentaire en haut du code.

Lien vers le code complet: [code/payload-decode.js](#)

Avec le décodeur complété nous pouvons maintenant voir en plus les informations du capteur (humidité et température):

Le payload reçu est initialement encodé sous forme binaire, ce qui correspond à une syntaxe de transfert optimale pour la transmission. Le décodeur JS nous permet d'avoir une syntaxe exploitable (JSON).

Test

Byte payload

CC 54 08 0D 01 78 01 7F FF 7F FF

FPort

1

Test decoder

Decoded test payload

```
{
  "data": {
    "battery_status": 3,
    "battery_voltage": 3156,
    "humidity": 37.6,
    "temp1": 20.61
  }
}
```

Figure 5: Résultats du décodeur complété

Accès au conteneur ALPHANET-DS

Après la connexion à la machine distante via SSH, nous avons mis à jour le système du conteneur, installé netcat et ouvert un serveur TCP sur le port 80 grâce à la commande `nc`.

Ensuite, depuis un navigateur web, nous sommes allés sur l'adresse `http://193.72.186.153:8087/script.php`. Nous voyons bien que le serveur ouvert précédemment a bien reçu une requête GET avec les entêtes client (User-Agent, Host, Accept, etc).

```
user-8@ds-tinf-ra-sf-8:~$ sudo nc -l -p 80
GET /script.php HTTP/1.1
Host: 193.72.186.153:8087
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:148.0) Gecko/20100101 Firefox/148.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: fr,fr-FR;q=0.9,en-US;q=0.8,en;q=0.7
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
DNT: 1
Sec-GPC: 1
Priority: u=0, i
```

Figure 6: Serveur TCP ouverture et réception

Intégration Webhook HTTP dans ALPHANET-DS

Installation de PHP et déploiement de l'application Nous avons installé PHP en mode standalone. Nous avons ensuite créé un répertoire `/html` afin d'héberger les fichiers de l'application. Nous avons ensuite lancé le serveur web intégré de PHP pour servir le contenu du répertoire courant.

Ce serveur minimal permet de tester rapidement une application.

Un script PHP de base nous a été fourni, nous l'avons déployé sur le conteneur à l'aide d'une commande `scp`.

Afin de simuler le comportement d'une future intégration avec TTN, nous avons testé notre script PHP en envoyant une requête HTTP POST.

Intégration Webhook avec TTN Sur le site TTN, nous avons créé une nouvelle intégration webhook qui renseigne le lien de notre script PHP.

En allant via un navigateur sur notre script, nous pouvons voir ce message dans notre serveur php, ce qui nous intéresse c'est le payload:

```

sans-fil/code on master [?] via v25.8.1 via v8.3.6
• > echo '{ "a": "b"}' | POST -E http://193.72.186.153:8087/script.php
POST http://193.72.186.153:8087/script.php
User-Agent: lwp-request/6.76 libwww-perl/6.76
Content-Length: 12
Content-Type: application/x-www-form-urlencoded

200 OK
Connection: close
Date: Thu, 23 Apr 2026 15:06:25 GMT
Host: 193.72.186.153:8087
Content-Type: text/html; charset=UTF-8
Client-Date: Thu, 23 Apr 2026 15:06:25 GMT
Client-Peer: 193.72.186.153:8087
Client-Response-Num: 1
X-Powered-By: PHP/8.2.30

```

Figure 7: Test serveur php minimal

```

[Thu Apr 30 08:53:16 2026] 34.255.49.188:21672 Accepted
{"end_device_ids":{"device_id":"ra-lht-65-8","application_ids":{"application_id":"ra-labo
3-a-8"},"dev_eui":"A840414881827810","join_eui":"4BA4A6DCC4B1C459"},"correlation_ids":["a
s:up:01KQEJKJ3XYC809TE7ZM39T664"],"rpc:/ttn.lorawan.v3.AppAs/SimulateUplink:45c68805-b1ee-
44f0-bffe-e26acf286879"},"received_at":"2026-04-30T06:53:16.285394516Z","uplink_message":
{"f_port":1,"frm_payload":"zFQIDQF4AX//f/8=","rx_metadata":[{"gateway_ids":{"gateway_id":
"test"},"rssi":42,"channel_rssi":42,"snr":4.2}],"settings":{"data_rate":{"lorawan":{"bandwid
th":125000,"spreading_factor":7}},"frequency":"868000000"},"simulated":true}
[Thu Apr 30 08:53:16 2026] 34.255.49.188:21672 [200]: POST /script.php
[Thu Apr 30 08:53:16 2026] 34.255.49.188:21672 Closing

```

Figure 8: Test webhook without payload formatter

Le payload ci-dessus reçu n'était pas exploitable directement. Nous avons toutefois déjà effectué des tests avec un payload formatter. Nous l'avons activé.

Maintenant, les données deviennent interprétables et peuvent être lues au format JSON.

```
[Thu Apr 30 09:35:57 2026] 34.255.49.188:34066 Accepted
{"end_device_ids":{"device_id":"ra-lht-65-8","application_ids":{"application_id":"ra-labo3-a-8"},"dev_eui":"A840414881827810",
"join_eui":"4BA4A6DCC4B1C459"},"correlation_ids":["as:up:01KQEN1PV7VFEBWAJ0919ZQV21","rpc:/ttn.lorawan.v3.AppAs/SimulateUplink
:fc94507f-221a-413e-a9ef-6fcffc080a07"],"received_at":"2026-04-30T07:35:57.030807453Z","uplink_message":{"f_port":1,"frm_paylo
ad":"zFQID0F4AX//f/8=","decoded_payload":{"data":{"battery_status":3,"battery_voltage":3156,"humidity":37.6,"temp1":20.61},"r
x_metadata":[{"gateway_ids":{"gateway_id":"test"},"rssi":42,"channel_rssi":42,"snr":4.2},"settings":{"data_rate":{"lora":{"ba
ndwidth":125000,"spreading_factor":7},"frequency":"868000000"},"simulated":true}
[Thu Apr 30 09:35:57 2026] 34.255.49.188:34066 [200]: POST /script.php
[Thu Apr 30 09:35:57 2026] 34.255.49.188:34066 Closing
```

Figure 9: Test webhook with payload formatter

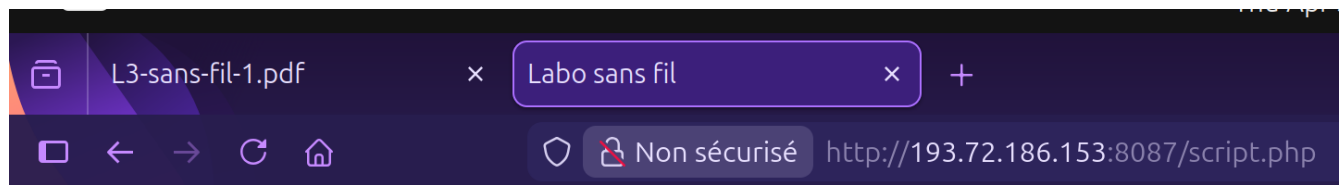
L'utilisation du webhook nous montre l'architecture client-serveur événementielle, où TTN agit comme un serveur client envoyant des requêtes POST vers notre serveur.

Affichage des données

Après la mise en place du webhook et du déploiement du script PHP sur le conteneur, nous avons mis en place l'affichage des données reçues.

Le script récupère les messages envoyés par TTN via une requête HTTP POST. Les données sont extraites depuis le payload JSON et stockées dans un fichier local afin de conserver la dernière valeur reçue.

Ces informations sont ensuite affichées sur une page web générée en PHP. Les valeurs de température, d'humidité et de batterie sont directement visibles dans le navigateur.



Données capteurs

Température: 10.9 °C

Humidité: 38.1 %

Batterie: 3049 mV

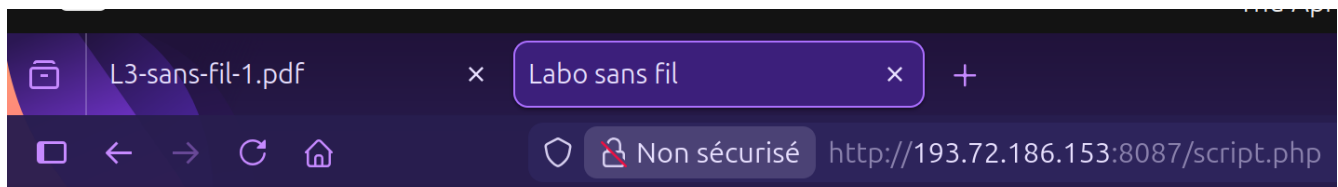
Figure 10: Affichage des données

La page se met à jour automatiquement toutes les 10 secondes afin de permettre un suivi quasi temps réel des données du capteur.

Gestion des alertes

En complément de l'affichage des données brutes, nous avons ajouté un système d'alertes afin d'interpréter les valeurs reçues et détecter des situations anormales.

Le script PHP récupère les données capteurs (température, humidité et tension de batterie) et les compare à des seuils définis. Si une valeur sort de la plage considérée comme normale, une alerte correspondante est générée.



Données capteurs

Température: 20.61 °C

Humidité: 37.6 %

Batterie: 3156 mV

Alertes

- Batterie faible (3156 mV)

Figure 11: Gestion des alertes

Les alertes générées sont ensuite affichées dynamiquement sur la page web sous forme de liste. Cela permet une lecture rapide de l'état du système sans analyser directement les valeurs numériques.

Le script PHP implémente la couche applicative de notre système IoT, responsable du traitement, stockage et visualisation des données.

Webhook JSON vs Protocol Buffers

Afin d'obtenir des logs exploitables, nous sommes revenus au script PHP de base et avons désactivé le payload formater.

Nous avons ensuite simulé un uplink depuis TTN afin d'observer la requête POST reçue par notre serveur. Dans un premier temps, nous avons utilisé le format JSON (format par défaut), puis nous avons testé le format Protocol Buffers.

En format JSON, les données sont lisibles et structurées. Le payload est présent dans le champ frm_payload. Bien que ce contenu ne soit pas directement interprétable, il reste facilement décodable et exploitable.

En revanche, en format Protocol Buffers, les données reçues apparaissent sous forme binaire. Elles ne sont pas lisibles dans le terminal et ne peuvent pas être traitées directement par le script PHP.

Nous en déduisons que le format JSON est plus adapté dans notre cas, car il permet une manipulation simple des données, contrairement au format Protocol Buffers qui nécessite un décodage spécifique. Cependant, ce dernier est plus compact et optimisé pour la transmission, ce qui le rend plus efficace côté machine.

Conclusion

Au cours de ce laboratoire, nous avons observé le fonctionnement d'un capteur LoRaWAN via TTN, notamment la réception des messages uplink et leur traitement.

Nous avons mis en place un décodeur permettant de transformer les données brutes en informations exploitables (température, humidité et batterie). Nous avons également utilisé un conteneur pour héberger un serveur recevant ces données via un webhook HTTP.

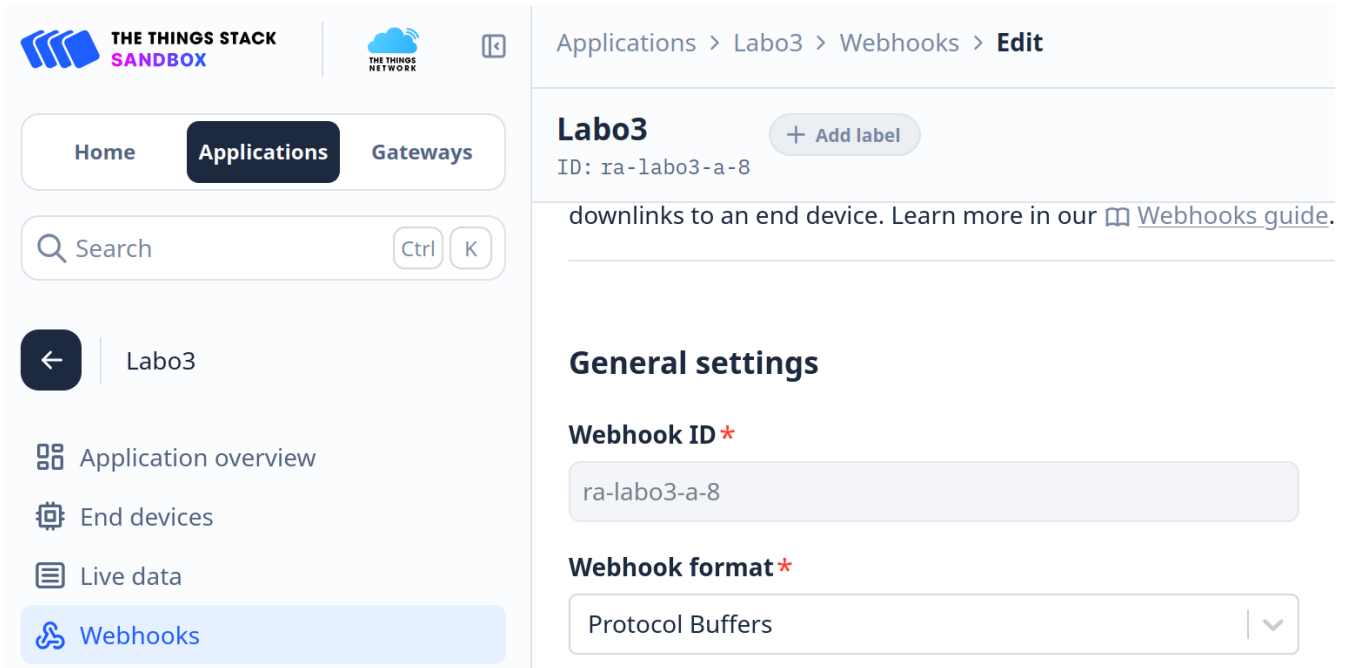


Figure 12: TTN changer le format du Webhook

```

user-8@ds-tinf-ra-sf-8:~/html$ sudo php -S 0.0.0.0:80
[Thu May 7 14:48:09 2026] PHP 8.2.30 Development Server (http://0.0.0.0:80) started
[Thu May 7 14:48:19 2026] 63.34.43.96:41991 Accepted
{"end_device_ids":{"device_id":"ra-lht-65-8","application_ids":{"application_id":"ra-labo3-a-8"},"dev_eui":"A840414881827810",
"join_eui":"4BA4A6DCC4B1C459"},"correlation_ids":["as:up:01KR17P08C3TXZ5N07KCQTFW4A"],"rpc:/ttn.lorawan.v3.AppAs/SimulateUplink
:2e74614e-5cd9-4e85-963c-de32a627d7a9"},"received_at":"2026-05-07T12:48:19.723557583Z","uplink_message":{"f_port":1,"frm_paylo
ad":"zFQIDQF4AX//f/8=","rx_metadata":{"gateway_ids":{"gateway_id":"test"},"rssi":42,"channel_rssi":42,"snr":4.2},"settings":
{"data_rate":{"lorawan":{"bandwidth":125000,"spreading_factor":7},"frequency":"868000000"},"simulated":true}
[Thu May 7 14:48:19 2026] 63.34.43.96:41991 [200]: POST /script.php
[Thu May 7 14:48:19 2026] 63.34.43.96:41991 Closing
[Thu May 7 14:48:43 2026] 63.34.43.96:54366 Accepted

1

ra-lht-65-8

ra-labo3-a-8@AH00xK00010Y as:up:01KR17QE756EE85EZ9ZPGS8AE4Mrpc:/ttn.lorawan.v3.AppAs/SimulateUplink:6c15118c-ee74-4ba1-8145-c
866e3626babb
000000pp:"
x002 0T
testE(BM(B)ff0@:
00 000
[Thu May 7 14:48:43 2026] 63.34.43.96:54366 [200]: POST /script.php
[Thu May 7 14:48:43 2026] 63.34.43.96:54366 Closing

```

Figure 13: Terninal JSON vs Protocol Buffers

Les données reçues ont ensuite été traitées par un script PHP, stockées localement, puis affichées sur une interface web avec une mise à jour automatique. Un système d'alertes a également été ajouté afin de signaler les valeurs anormales.

Enfin, nous avons comparé les formats JSON et Protocol Buffers pour la transmission des données. Nous avons constaté que le format JSON est directement exploitable dans notre cas, tandis que le format Protocol Buffers nécessite un traitement supplémentaire car il est binaire.

Reponse aux questions du compte-rendu

1. Relier deux LAN Ethernet via un bridge WIFI

a) Est-ce normal ? Que se passe-t-il probablement ? Quel est le risque éventuel ?

Est-ce normal? Non. Un vrai bridge en couche 2 devrait relayer les trames mais conserver l'adresse MAC source originale de l'expéditeur.

Que se passe-t-il probablement? Les access points en mode "bridge" fonctionnent plutôt en mode **relais avec réécriture MAC** (MAC rewriting/source MAC replacement):

1. Du côté LAN1: L'Access point reçoit une trame d'une machine (ex: MAC_A)
2. En transit: Au lieu de conserver MAC_A, l'AP **remplace l'adresse MAC source par sa propre adresse MAC** (MAC_AP1)
3. Du côté LAN2: Toutes les machines du LAN1 apparaissent avec l'adresse MAC de l'AP du LAN1

Ce qui expliquerait pourquoi toutes les machines du LAN1 possèdent la même adresse : ce serait celle de l'Access point.

Cependant, tout fonctionne "normalement" car TCP/IP utilise les **adresses IP** (couche 3), pas les adresses MAC, pour le routage général. On aura des problèmes en cas d'utilisation de protocole dépendant des adresses MAC.

Risques éventuels? Perte d'information d'adressage - Il est impossible d'identifier la véritable machine source du LAN1 à partir du LAN2. - Les adresses MAC originales sont perdues.

Problèmes avec ARP - Les tables ARP ne reflètent pas la topologie réelle - Une machine du LAN2 qui veut faire un ping à une machine du LAN1 doit passer par l'AP, pas directement. Il y a donc un risque d'incohérence ARP si les réponses ARP ne sont pas cohérentes avec le flux de données

Problèmes de sécurité - Certains filtres / politiques de sécurité ne s'exécuteraient pas, car on pourrait penser que tout vient du même AP à cause de l'adresse MAC identique

b) Consultation du format de trame 802.11, explication de comment les 4 adresses devraient être utilisées, si cette fonction n'était pas activée

Une trame 802.11 contient jusqu'à **4 champs d'adresse MAC**, dont l'utilisation dépend de deux flags : - **To DS** (To Distribution System) : 1 = la trame est destinée au système de distribution (AP) - **From DS** (From Distribution System) : 1 = la trame provient du système de distribution (AP)

Utilisation des 4 adresses selon To DS / From DS :

To DS	From DS	Addr1	Addr2	Addr3	Addr4	Contexte
0	0	DA	SA	BSSID	N/A	Ad-hoc, infra client→AP
0	1	DA	BSSID	SA	N/A	AP → client
1	0	BSSID	SA	DA	N/A	Client → AP (distribution)
1	1	Rcv AP	Transmit AP	DA	SA	WDS/Bridge ← Mode bridge transparent



Cela est nécessaire par le fait que c’est la seule façon de détecter, à posteriori, des collisions et donc des destructions de trame.

(exception : option de suppression des confirmations en 802.11e)

9 Format de trame 802.11

Format de trame (taille en octets)

Préambule	Entête PLCP	Données couche MAC et supérieures	CRC
-----------	-------------	-----------------------------------	-----

Données couche MAC et supérieures

FC (2)	durée/ID (2)	adresse 1 (6)	adr. 2 (6)	ad. 3 (6)	séquence (2)	adr. 4 (6)	données (0-2312)	CRC (4)
--------	--------------	---------------	------------	-----------	--------------	------------	------------------	---------

Détail sur le champ FC (taille en bits)

version (2 ; 00)	type (2)	sous-type (4)
------------------	----------	---------------

flags :

to DS (1)	from DS (1)	more frag (1)	retry (1)	pwr mgt (1)	more data (1)	WEP (1)	order (1)
-----------	-------------	---------------	-----------	-------------	---------------	---------	-----------

9.1 Notes

Figure 14: Format d’une trame 802.11 venant du support de cours “technologies-wifi

On remarque que le mode “ad-hoc” est utilisé quand les deux champs “To DS” et “from DS” sont à 0, et que le mode bridge se passe quand les deux champs sont à 1.

C’est le **mode WDS/bridge transparent** qui devrait être utilisé pour un vrai bridge 802.11. Cela permettrait de préserver à la fois l’identité des APs (sans fil) et l’identité des machines Ethernet originales (câblées).

Ce n’est pas le cas avec la configuration utilisée ici : les APs simples en mode “bridge” n’utilisent pas le mode 4 adresses, mais le mode **(1,0)**.

2. exemple de secret partagé qui permette d’atteindre 256 bits

L’entropie d’un mot passe choisi parmi 94 caractères ASCII imprimables, choisis aléatoirement et sur une longueur de 40 caractères, vaut:

$$H = 40 * \log(94) \approx 262bits$$

Un exemple de secret partagé robuste serait de générer 40 caractères aléatoires parmi un alphabet qui serait plutôt large. Voici un exemple: vT7!qZ@2Lm#8xR\$4Pw9&Ks1^Nd6*Hy3+Uf0!Cb

Exemple “parfait” Le meilleur cas serait d’utiliser des caractères hexadécimaux pour représenter une clé purement aléatoire de 256 bits. Chaque caractère représente 4 bits:

$$64 * 4 = 256bits$$

On arrive à exactement 256 bits d’entropie.

avancés : comment peut-on, si un mot de passe n'a pas une entropie suffisante, stretch 10 la clé pour améliorer la situation On peut appliquer des milliers/millions d'itérations, parfois avec coût mémoire élevé, pour dériver une clé AES à partir du mot de passe. Cela ralentit fortement les attaques par brute force car il augmente le coût de calcul pour l'attaquant.

3. qu'est-ce qu'un portail captif dans le contexte de la sécurisation d'un réseau sans-fil sans chiffrement ? y-a-t-il des problèmes avec cette stratégie ?

C'est une page web d'authentification s'affichant automatiquement, imposant des conditions (connexion, acceptation des conditions) avant de pouvoir accéder à un réseau sans fil.

Cependant, un problème reste: le wifi reste non chiffré, même si une authentification est requise. Les trames peuvent être interceptées et un attaquant proche pourrait écouter le trafic.

4. faites un schéma couche 3 et couche 2 du pont WiFi dans la variante où les équipements réseau sont administrés dans un VLAN séparé, placez un PC d'administration et indiquez quels ports sont en trunk et quel port est en access

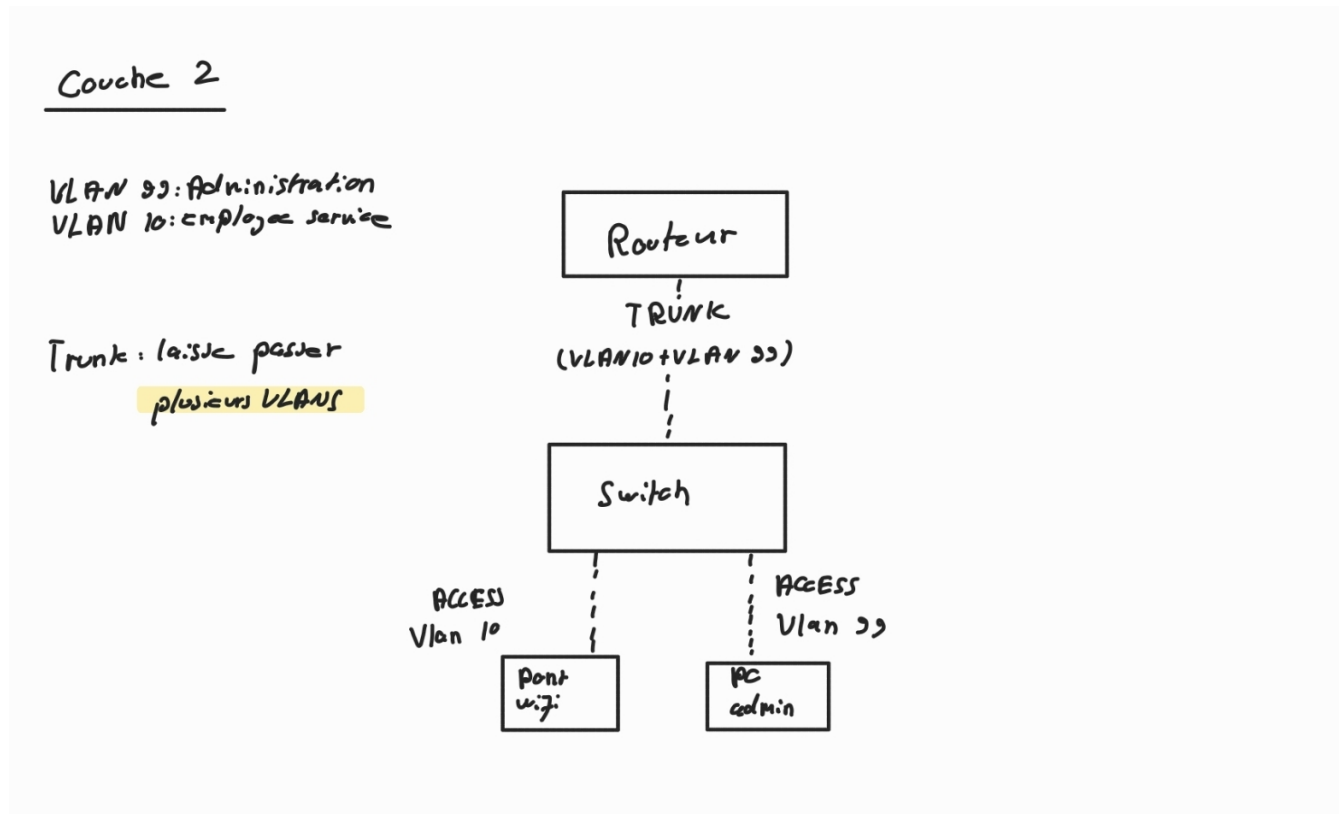


Figure 15: Schéma en couche 2

5. avec deux antennes à gain 30 dB, quelle distance pourriez-vous atteindre ? quel serait le problème de cette liaison ?

On pourrait atteindre plusieurs dizaines de kilomètres, mais cela dépend de certains facteurs: * Le faisceau à ce niveau de dB est très étroit, il faut être sûr de bien les avoir alignés * Avoir une ligne de vue dégagée (zone de Fresnel).

Problème possible: la puissance isotrope rayonnée équivalente (EIRP) est souvent réglementée selon les pays.

$$EIRP = P_{\text{émission}} + G_{\text{antenne}}$$

Couche 3

VLAN 10 (IoT / wifi: capteurs)

Gateway: 192.168.10.1

Réseau: 192.168.10.0/24

Capteurs wifi

192.168.10.x

Bridge wifi: (IP: 192.168.10.2)

Routeur / Interface VLAN 10
192.168.10.1

VLAN 99 (admin)

Réseau: 192.168.99.0/24

Gateway: 192.168.99.1

PC admin: 192.168.99.10
|
|

Routeur / Interface VLAN 99
192.168.99.1

Figure 16: Schéma en couche 3

Si on suppose : * Wi-Fi 5 GHz, * puissance émission = 20 dBm, * antennes = 30 dBi chacune, * sensibilité réception = env. -75 dBm.

Si on a 30 dB de gains, on arrive à 80 (30+30+20), ce qui dépasse les limites légales. Il est alors nécessaire de réduire la puissance d'émission.

Problème de la liaison :

- Risques d'interférence (perturbations radio, météo, arbres)
- Limites légales.

6. donnez des arguments pour l'exploitation en mode bridge et en mode routeur au sein de l'entreprise (les réseaux filaires et sans-fils sont respectivement switchés et routés)

Bridge

En mode Bridge, les utilisateurs sans fil et filaire sont dans le même réseau de couche 2.

Avantages: * Partage facile, par exemple avec une imprimante * Même plan d'adressage IP * Broadcast

Désavantages: * Sécurité plus faible entre wifi et filaire * Grand domaine de broadcast * Propagation possible (tempête broadcast, attaques ARP)

Mode routeur En mode routeur, le wifi est séparé du réseau filaire. Les deux sous-réseaux sont alors bien distincts (sans fil, réseau filaire routé)

Avantages: * Meilleure sécurité * Isolation des clients Wi-fi * Filtrage avec firewall possible * Politiques réseaux différentes selon les utilisateurs (employés, invités, admin)

Désavantages: * Configuration plus complexe * Il faut gérer le DHCP, le routage, le firewall et le VLAN.

7. Problème: des trames se perdent

- 802.11e garantit la **QoS** pour des données telles que la voix durant les appels par exemple.
- 802.11n garantit le **débit haut**.

Problème possibles: * Le QoS est priorisé et les paquets à priorité basse peuvent être mis en attente si un trafic prioritaire passe avant eux. * Le wifi est un réseau à accès concurrent. Il est possible que plusieurs stations se disputent le canal. * Il y a eu des retransmissions: Si une trame doit être retransmise, le délai devient imprévisible et les paquets arrivent de manière irrégulière. Il y a alors moins de coupures mais une forte latence est possible. * Le 802.11n peut empirer la situation : il améliore le débit global mais rend le comportement plus variable et peut amplifier les effets des interférences.

8. Améliorer la sécurité d'un réseau de capteurs codés en 802.11 dont le chip wifi ne supporte pas le chiffrement en couche 2

Soit un réseau de capteurs que vous avez codés, en 802.11. Vous remarquez, trop tard, que le chip WiFi embarqué ne supporte pas le chiffrement en couche 2 (ou alors du vieux WEP à 64 bits cassable en quelques secondes). Que pouvez-vous faire pour améliorer la sécurité ? Pourquoi ce n'est pas idéal dans ce cas ?

On pourrait : * Utiliser du chiffrement dans la couche application (TLS/HTTPS, MQTTs) * Chiffrer directement les données envoyées par les capteurs avant leur transmission * Mettre en place une authentification plus forte entre les capteurs et le serveur.

En agissant ainsi, même si le wifi était intercepté, les données resteraient difficiles à lire sans la clé de chiffrement.

Ce n'est pas idéal, car le réseau Wifi reste ouvert dans la couche 2 (liaison). En effet, sans chiffrement en couche 2, toutes les trames wifi sont visibles. De plus, les capteurs IOT ont souvent peu de ressources. Le chiffrement applicatif en demande beaucoup.

La gestion des clés devient aussi plus complexe : le stockage, la distribution et le renouvellement de manière sécurisée complique l'architecture.

Il manquera également des protections du Wifi moderne (WPA2/3 fournissent de l'authentification, l'intégrité des trames, et la protection contre certaines attaques réseau). Si on met en place un chiffrement uniquement applicatif, on ne couvre pas entièrement ces aspects.

9. Listez les mesures à prendre pour sécuriser un réseau Wifi et classez-les de la meilleure à la moins utile.

1. Chiffrement WPA/WPA2
2. Utiliser un mot de passe de connexion fort
3. Faire les mises à jour
4. Séparation des réseaux avec des VLAN
5. Désactiver WPS (pratique mais vulnérable)
6. Portail captif

Annexes

[payload-decode.js](#)

[script.php](#)

[VLAN ids et configurations trunk](#)

Utilisation de l'IA

- Code: idées, aide à la correction
- Formules: copié-coller des formules de base, aide à la correction des résultats

Sources

[802.11n et 802.11e definitions](#)

[Portail captif définition](#)